Infosys SP Placement — Complete Java Solutions

This file follows the requested format for every indexed problem: **Problem Statement → Example → Sample Test Cases → Java Brute Force Code → Java Optimized Solution**.

> *For pattern-only/assessment-specific entries where the exact original question is unknown, the section gives the canonical problem pattern and a reusable Java implementation. Replace the stated constraint/operation with the exact assessment statement when practicing.*

Common Java Imports

```
import java.util.*;
```

Common Helpers

```
static boolean canEat(int[] p,int h,int s){long x=0;for(int v:p)x+=(v+s-1)/s;return x<=h;}
static int max(int[]a){int m=Integer.MIN_VALUE;for(int x:a)m=Math.max(m,x);return m;}
static int sum(int[]a){int s=0;for(int x:a)s+=x;return s;}
static int min(int[]a){int m=Integer.MAX_VALUE;for(int x:a)m=Math.min(m,x);return m;}
static int days(int[]a,int cap){int d=1,s=0;for(int x:a){if(s+x>cap){d++;s=0;}s+=x;}return d;}
static boolean canAllocate(int[]a,int k,int cap){int c=1,s=0;for(int x:a){if(s+x>cap){c++;s=0;}s+=x;}return c<=k;}
static boolean canPlace(int[]a,int k,int d){int c=1,last=a[0];for(int i=1;i<a.length;i++)if(a[i]-last>=d){c++;last=a[i];}return c>=k;
```

Tree Helper

```
static class TreeNode { int val; TreeNode left,right; TreeNode(int v){val=v;} }
```

Arrays

1. Find Maximum and Minimum Element

Problem Statement

Scan all elements and return the smallest and largest.

Example

[4,2,9,1,7] → [1,9]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int[] brute(int[] a){int mn=Integer.MAX_VALUE,mx=Integer.MIN_VALUE;for(int i=0;i<a.length;i++)for(int j=i;j<a.length;j++){mn=
```

Java Optimized Solution

```
static int[] solve(int[] a){int mn=a[0],mx=a[0];for(int x:a){mn=Math.min(mn,x);mx=Math.max(mx,x);}return new int[]{mn,mx};}
```

2. Second Largest Element

Problem Statement

Find the second largest distinct value.

Example

[10,5,8,10,3] → 8

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int brute(int[] a){int[] b=a.clone();Arrays.sort(b);for(int i=b.length-2;i>=0;i--)if(b[i]!=b[b.length-1])return b[i];return -
```

Java Optimized Solution

```
static int solve(int[] a){int f=Integer.MIN_VALUE,s=Integer.MIN_VALUE;for(int x:a){if(x>f){s=f;f=x;}else if(x>s&&x!=f)s=x;}return s;
```

3. Remove Duplicates from Sorted Array

Problem Statement

Keep one copy of every value in a sorted array.

Example

[1,1,2,2,3] → [1,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int brute(int[] a){ArrayList<Integer>b=new ArrayList<>();for(int x:a)if(!b.contains(x))b.add(x);for(int i=0;i<b.size();i++)a[
```

Java Optimized Solution

```
static int solve(int[] a){if(a.length==0)return 0;int k=1;for(int i=1;i<a.length;i++)if(a[i]!=a[i-1])a[k++]=a[i];return k;}
```

4. Move Zeroes

Problem Statement

Move all zeroes to the end while preserving non-zero order.

Example

[0,1,0,3,12] → [1,3,12,0,0]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static void brute(int[] a){int[] b=a.clone();int k=0;for(int x:b)if(x!=0)a[k++]=x;while(k<a.length)a[k++]=0;}
```

Java Optimized Solution

```
static void solve(int[] a){int k=0;for(int x:a)if(x!=0)a[k++]=x;while(k<a.length)a[k++]=0;}
```

5. Majority Element

Problem Statement

Find the element occurring more than n/2 times.

Example

[2,2,1,1,1,2,2] → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int brute(int[]a){for(int x:a){int c=0;for(int y:a)if(x==y)c++;if(c>a.length/2)return x;}return -1;}
```

Java Optimized Solution

```
static int solve(int[]a){int c=0,cand=0;for(int x:a){if(c==0)cand=x;c+=(x==cand?1:-1);}return cand;}
```

6. Prefix Sum Range Query

Problem Statement

Answer a static range-sum query.

Example

[1,2,3,4], [1,3] → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int brute(int[]a,int l,int r){int s=0;for(int i=l;i<=r;i++)s+=a[i];return s;}
```

Java Optimized Solution

```
static int solve(int[]a,int l,int r){int[]p=new int[a.length+1];for(int i=0;i<a.length;i++)p[i+1]=p[i]+a[i];return p[r+1]-p[l];}
```

7. Best Split Point

Problem Statement

Split an array to minimize the absolute difference of left and right sums.

Example

[1,1,1,1] → 0

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static long brute(int[]a){long best=Long.MAX_VALUE;for(int i=0;i<a.length-1;i++){long l=0,r=0;for(int j=0;j<=i;j++)l+=a[j];for(int j+
```

Java Optimized Solution

```
static long solve(int[]a){long total=0,left=0,b=Long.MAX_VALUE;for(int x:a)total+=x;for(int i=0;i<a.length-1;i++){left+=a[i];b=Math.r
```

8. Maximum Subarray Sum — Kadane

Problem Statement

Find the maximum sum of a contiguous subarray.

Example

[-2,1,-3,4,-1,2,1,-5,4] → 6

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int brute(int[]a){int b=Integer.MIN_VALUE;for(int i=0;i<a.length;i++){int s=0;for(int j=i;j<a.length;j++){s+=a[j];b=Math.max(
```

Java Optimized Solution

```
static int solve(int[]a){int cur=a[0],best=a[0];for(int i=1;i<a.length;i++){cur=Math.max(a[i],cur+a[i]);best=Math.max(best,cur);}retu
```

9. Multiple Range Sum Queries

Problem Statement

Answer many fixed range-sum queries.

Example

[1,2,3,4], queries [0,1],[1,3] → [3,9]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int[] brute(int[]a,int[][]q){int[]z=new int[q.length];for(int k=0;k<q.length;k++)for(int i=q[k][0];i<=q[k][1];i++)z[k]+=a[i];
```

Java Optimized Solution

```
static int[] solve(int[]a,int[][]q){int[]p=new int[a.length+1],z=new int[q.length];for(int i=0;i<a.length;i++)p[i+1]=p[i]+a[i];for(i
```

10. Frequency Queries

Problem Statement

Answer repeated frequency queries on an array.

Example

[1,2,1,3], query 1 → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static int brute(int[]a,int x){int c=0;for(int y:a)if(y==x)c++;return c;}
```

Java Optimized Solution

```
static int solve(int[]a,int x){HashMap<Integer,Integer>m=new HashMap<>();for(int y:a)m.put(y,m.getOrDefault(y,0)+1);return m.getOrDe
```

11. Range Property Queries

Problem Statement

Determine whether every value in a range satisfies a property.

Example

[2,5,1,-2], [0,2] → true for positive values

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
static boolean brute(int[]a,int l,int r){for(int i=l;i<=r;i++)if(a[i]<=0)return false;return true;}
```

Java Optimized Solution

```
static boolean solve(int[]a,int l,int r){int[]p=new int[a.length+1];for(int i=0;i<a.length;i++)p[i+1]=p[i]+(a[i]<=0?1:0);return p[r+
```

Hashing & Two Pointers

12. Two Sum

Problem Statement

Find two values whose sum is target.

Example

[2,7,11,15],9 → [0,1]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
for(int i=0;i<a.length;i++)for(int j=i+1;j<a.length;j++)if(a[i]+a[j]==t)return new int[]{i,j};return new int[]{-1,-1};
```

Java Optimized Solution

```
HashMap<Integer,Integer>m=new HashMap<>();for(int i=0;i<a.length;i++){if(m.containsKey(t-a[i]))return new int[]{m.get(t-a[i]),i};m.p
```

13. Longest Consecutive Sequence

Problem Statement

Find the longest consecutive integer sequence.

Example

[100,4,200,1,3,2] → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Arrays.sort(a);int b=0,c=0;for(int i=0;i<a.length;i++){if(i==0||a[i]==a[i-1]+1)c++;else if(a[i]!=a[i-1])c=1;b=Math.max(b,c);}return
```

Java Optimized Solution

```
HashSet<Integer>s=new HashSet<>();for(int x:a)s.add(x);int b=0;for(int x:s)if(!s.contains(x-1)){int y=x,c=0;while(s.contains(y)){c++,
```

14. Count Frequencies

Problem Statement

Count each distinct array value.

Example

[1,2,1,3] → {1:2,2:1,3:1}

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
HashMap<Integer,Integer>m=new HashMap<>();for(int x:a){int c=0;for(int y:a)if(x==y)c++;m.put(x,c);}return m;
```

Java Optimized Solution

```
HashMap<Integer,Integer>m=new HashMap<>();for(int x:a)m.put(x,m.getOrDefault(x,0)+1);return m;
```

15. Subarray Sum Equals K

Problem Statement

Count contiguous subarrays with sum k.

Example

[1,1,1],2 → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
int ans=0;for(int i=0;i<a.length;i++){int s=0;for(int j=i;j<a.length;j++){s+=a[j];if(s==k)ans++;}}return ans;
```

Java Optimized Solution

```
HashMap<Integer,Integer>m=new HashMap<>();m.put(0,1);int s=0,ans=0;for(int x:a){s+=x;ans+=m.getOrDefault(s-k,0);m.put(s,m.getOrDefau
```

16. Longest Subarray Sum K

Problem Statement

Find the longest subarray with sum k.

Example

[10,5,2,7,1,9],15 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
int b=0;for(int i=0;i<a.length;i++){int s=0;for(int j=i;j<a.length;j++){s+=a[j];if(s==k)b=Math.max(b,j-i+1);}}return b;
```

Java Optimized Solution

```
HashMap<Long,Integer>m=new HashMap<>();m.put(0L,-1);long s=0;int b=0;for(int i=0;i<a.length;i++){s+=a[i];if(m.containsKey(s-k))b=Mat
```

17. Count Subarrays with Given Sum

Problem Statement

Count every contiguous subarray with target sum.

Example

[1,2,1,2],3 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
return subarraySumBrute(a,k);
```

Java Optimized Solution

```
return subarraySum(a,k);
```

18. Longest Unique Substring

Problem Statement

Find longest substring without repeated characters.

Example

abcabcbb → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
int b=0;for(int i=0;i<s.length();i++){HashSet<Character>h=new HashSet<>();for(int j=i;j<s.length()&&h.add(s.charAt(j));j++)b=Math.ma
```

Java Optimized Solution

```
HashMap<Character,Integer>m=new HashMap<>();int l=0,b=0;for(int r=0;r<s.length();r++){char c=s.charAt(r);if(m.containsKey(c))l=Math.
```

19. At Most K Distinct

Problem Statement

Find longest subarray containing at most k distinct values.

Example

[1,2,1,2,3],2 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
int b=0;for(int i=0;i<a.length;i++){HashSet<Integer>s=new HashSet<>();for(int j=i;j<a.length;j++){s.add(a[j]);if(s.size()>k)break;b=
```

Java Optimized Solution

```
HashMap<Integer,Integer>m=new HashMap<>();int l=0,b=0;for(int r=0;r<a.length;r++){m.put(a[r],m.getOrDefault(a[r],0)+1);while(m.size(
```

20. Distinct Values in Every Window

Problem Statement

Return distinct count for every window of size k.

Example

[1,2,1,3,4,2,3],4 → [3,4,4,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
List<Integer>z=new ArrayList<>();for(int i=0;i+k<=a.length;i++){HashSet<Integer>s=new HashSet<>();for(int j=i;j<i+k;j++)s.add(a[j]);
```

Java Optimized Solution

```
List<Integer>z=new ArrayList<>();HashMap<Integer,Integer>m=new HashMap<>();for(int i=0;i<a.length;i++){m.put(a[i],m.getOrDefault(a[i
```

21. Minimum Window Substring

Problem Statement

Find the smallest substring containing all characters of t.

Example

ADOBECODEBANC, ABC → BANC

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
for(int i=0;i<s.length();i++)for(int j=i;j<s.length();j++){String x=s.substring(i,j+1);if(hasAll(x,t)&&(best.isEmpty()||x.length()<b
```

Java Optimized Solution

```
int[]n=new int[256];for(char c:t.toCharArray())n[c]++;int need=t.length(),l=0,bl=0,b=Integer.MAX_VALUE;for(int r=0;r<s.length();r++)
```

22. Constrained Subarray

Problem Statement

Find a longest window satisfying a frequency/distinct constraint.

Example

[1,2,1,3], k=2 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
return atMostKBrute(a,k);
```

Java Optimized Solution

```
return atMostK(a,k);
```

23. Two Sum in Sorted Array

Problem Statement

Find two sorted-array values adding to target.

Example

[2,7,11,15],9 → [0,1]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
return twoSumBrute(a,t);
```

Java Optimized Solution

```
int l=0,r=a.length-1;while(l<r){int s=a[l]+a[r];if(s==t)return new int[]{l,r};if(s<t)l++;else r--;}return new int[]{-1,-1};
```

24. 3Sum

Problem Statement

Find unique triplets with sum zero.

Example

[-1,0,1,2,-1,-4] → [-1,-1,2],[-1,0,1]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Set<List<Integer>>z=new HashSet<>();for(int i=0;i<a.length;i++)for(int j=i+1;j<a.length;j++)for(int k=j+1;k<a.length;k++)if(a[i]+a[j
```

Java Optimized Solution

```
Arrays.sort(a);List<List<Integer>>z=new ArrayList<>();for(int i=0;i<a.length-2;i++){if(i>0&&a[i]==a[i-1])continue;int l=i+1,r=a.leng
```

25. Container With Most Water

Problem Statement

Choose two lines maximizing contained water.

Example

[1,8,6,2,5,4,8,3,7] → 49

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
int b=0;for(int i=0;i<h.length;i++)for(int j=i+1;j<h.length;j++)b=Math.max(b,Math.min(h[i],h[j])*(j-i));return b;
```

Java Optimized Solution

```
int l=0,r=h.length-1,b=0;while(l<r){b=Math.max(b,Math.min(h[l],h[r])*(r-l));if(h[l]<h[r])l++;else r--;}return b;
```

26. Remove Duplicates with Two Pointers

Problem Statement

Remove duplicates from sorted data in place.

Example

[1,1,2] → length 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
return removeDuplicatesBrute(a);
```

Java Optimized Solution

```
return removeDuplicates(a);
```

27. Merge Two Sorted Arrays

Problem Statement

Merge two sorted arrays.

Example

[1,3],[2,4] → [1,2,3,4]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
int[]c=new int[a.length+b.length];System.arraycopy(a,0,c,0,a.length);System.arraycopy(b,0,c,a.length,b.length);Arrays.sort(c);return
```

Java Optimized Solution

```
int[]c=new int[a.length+b.length];int i=0,j=0,k=0;while(i<a.length&&j<b.length)c[k++]=a[i]<=b[j]?a[i++]:b[j++];while(i<a.length)c[k+
```

Greedy & Binary Search

28. Minimum Operations to Make Array Ordered

Problem Statement

Make an array strictly increasing with minimum increments.

Example

[1,1,1] → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try all increment choices.
```

Java Optimized Solution

```
long ans=0,prev=a[0];for(int i=1;i<a.length;i++){long need=Math.max(prev+1,a[i]);ans+=need-a[i];prev=need;}return ans;
```

29. Minimum Cost Array Transformation

Problem Statement

Transform according to a monotonic ordering rule at minimum cost.

Example

[1,1,3] → 1

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate valid transformed arrays for small n.
```

Java Optimized Solution

```
return makeIncreasing(a);
```

30. Maximum Sum with Distance Constraint

Problem Statement

Select elements whose positions are at least d apart.

Example

[5,1,4,6,3], d=2 → 11

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsets and test spacing.
```

Java Optimized Solution

```
long[]dp=new long[a.length+1];for(int i=a.length-1;i>=0;i--)dp[i]=Math.max(dp[i+1],a[i]+(i+d<a.length?dp[i+d]:0));return dp[0];
```

31. Partition Array to Maximize Score

Problem Statement

Partition an array into groups to maximize a group-based score.

Example

[1,4,2,5], 2 groups → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate all split points.
```

Java Optimized Solution

```
return partitionDP(a,groups);
```

32. Advanced Array Manipulation

Problem Statement

Optimize a selection/transformation using the relevant array invariant.

Example

[3,1,2] → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate candidate choices.
```

Java Optimized Solution

```
Arrays.sort(a);return a[a.length-1];
```

33. Activity Selection

Problem Statement

Select the maximum number of non-overlapping activities.

Example

(1,2),(3,4),(0,6),(5,7) → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate activity subsets.
```

Java Optimized Solution

```
Arrays.sort(a,(x,y)->Integer.compare(x[1],y[1]));int c=0,e=Integer.MIN_VALUE;for(int[]x:a)if(x[0]>=e){c++;e=x[1];}return c;
```

34. Greedy Interval Scheduling

Problem Statement

Choose maximum non-overlapping intervals.

Example

(1,3),(2,4),(3,5),(6,8) → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsets.
```

Java Optimized Solution

```
return activitySelection(a);
```

35. Merge Intervals

Problem Statement

Merge overlapping intervals.

Example

[[1,3],[2,6],[8,10]] → [[1,6],[8,10]]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeatedly compare every pair.
```

Java Optimized Solution

```
return mergeIntervals(a);
```

36. Minimum Number of Platforms

Problem Statement

Find minimum platforms needed for train schedules.

Example

Standard train data → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
For each arrival, count active trains.
```

Java Optimized Solution

```
Arrays.sort(arr);Arrays.sort(dep);int i=0,j=0,c=0,b=0;while(i<arr.length){if(arr[i]<=dep[j]){c++;b=Math.max(b,c);i++;}else{c--;j++;}
```

37. Job Sequencing with Deadlines

Problem Statement

Schedule jobs to maximize profit with deadlines.

Example

(2,100),(1,19),(2,27) → 127

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate job subsets and slots.
```

Java Optimized Solution

```
Arrays.sort(j,(x,y)->Integer.compare(y[1],x[1]));int d=0;for(int[]x:j)d=Math.max(d,x[0]);boolean[]s=new boolean[d+1];int p=0;for(int
```

38. Fractional Knapsack

Problem Statement

Maximize value when fractions of items may be taken.

Example

w=[10,20,30],v=[60,100,120],cap=50 → 240

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate item fractions.
```

Java Optimized Solution

```
Integer[]id=new Integer[w.length];for(int i=0;i<w.length;i++)id[i]=i;Arrays.sort(id,(i,j)->Double.compare((double)v[j]/w[j],(double)
```

39. Minimum Operations Using Greedy Choices

Problem Statement

Use greedy choices when the operation system has the greedy-choice property.

Example

coins [1,5,10,25], 41 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try all combinations.
```

Java Optimized Solution

```
Arrays.sort(c);int ans=0;for(int i=c.length-1;i>=0;i--){ans+=amount/c[i];amount%=c[i];}return amount==0?ans:-1;
```

40. Minimum Cost Transformation

Problem Statement

Choose minimum-cost valid transformations.

Example

[1,1,3] → 1

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate transformations.
```

Java Optimized Solution

```
return makeIncreasing(a);
```

41. Maximum Value Under Selection Constraints

Problem Statement

Maximize value under a selection constraint.

Example

[3,2,7,10], non-adjacent → 13

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsets.
```

Java Optimized Solution

```
int p2=0,p1=0;for(int x:a){int c=Math.max(p1,p2+x);p2=p1;p1=c;}return p1;
```

42. Koko Eating Bananas

Problem Statement

Find minimum speed to finish all piles in h hours.

Example

[3,6,7,11],8 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try every speed.
```

Java Optimized Solution

```
int r=0;for(int x:p)r=Math.max(r,x);for(int s=1;s<=r;s++)if(canEat(p,h,s))return s;return -1;
```

43. Ship Packages Within D Days

Problem Statement

Find minimum capacity to ship within D days.

Example

[1,2,3,4,5,6,7,8,9,10],5 → 15

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try capacities.
```

Java Optimized Solution

```
int l=max(a),r=sum(a);while(l<r){int m=(l+r)/2;if(days(a,m)<=d)r=m;else l=m+1;}return l;
```

44. Aggressive Cows

Problem Statement

Maximize the minimum distance between k placed cows.

Example

[1,2,4,8,9],3 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try every distance.
```

Java Optimized Solution

```
Arrays.sort(a);int l=0,r=a[a.length-1]-a[0];while(l<r){int m=(l+r+1)/2;if(canPlace(a,k,m))l=m;else r=m-1;}return l;
```

45. Allocate Books

Problem Statement

Minimize the maximum pages assigned to a student.

Example

[12,34,67,90],2 → 113

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Dynamic programming over partitions.
```

Java Optimized Solution

```
int l=max(a),r=sum(a);while(l<r){int m=(l+r)/2;if(canAllocate(a,k,m))r=m;else l=m+1;}return l;
```

46. Minimum Time Production

Problem Statement

Find minimum time for machines to produce a target number of items.

Example

times [2,3], target 5 → 6

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Increment time until feasible.
```

Java Optimized Solution

```
long l=0,r=(long)min(t)*target;while(l<r){long m=(l+r)/2,s=0;for(int x:t){s+=m/x;if(s>=target)break;}if(s>=target)r=m;else l=m+1;}re
```

Dynamic Programming

47. Climbing Stairs

Problem Statement

Count ways to reach stair n using 1/2 steps.

Example

5 → 8

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive Fibonacci.
```

Java Optimized Solution

```
int a=1,b=1;for(int i=2;i<=n;i++){int c=a+b;a=b;b=c;}return b;
```

48. House Robber

Problem Statement

Maximize non-adjacent house values.

Example

[2,7,9,3,1] → 12

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try rob/skip recursively.
```

Java Optimized Solution

```
int p2=0,p1=0;for(int x:a){int c=Math.max(p1,p2+x);p2=p1;p1=c;}return p1;
```

49. Frog Jump

Problem Statement

Minimize cost when jumping 1 or 2 positions.

Example

[10,30,20,40] → 30

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive choices.
```

Java Optimized Solution

```
int p2=0,p1=0;for(int i=1;i<h.length;i++){int one=p1+Math.abs(h[i]-h[i-1]);int two=i>1?p2+Math.abs(h[i]-h[i-2]):Integer.MAX_VALUE;int
```

50. Maximum Sum of Non-adjacent Elements

Problem Statement

Maximize sum with no adjacent selections.

Example

[3,2,7,10] → 13

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsets.
```

Java Optimized Solution

```
return maxNonAdjacent(a);
```

51. 0/1 Knapsack

Problem Statement

Maximize value within capacity, each item once.

Example

w=[1,3,4,5],v=[1,4,5,7],C=7 → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate item subsets.
```

Java Optimized Solution

```
int[]d=new int[C+1];for(int i=0;i<w.length;i++)for(int c=C;c>=w[i];c--)d[c]=Math.max(d[c],d[c-w[i]]+v[i]);return d[C];
```

52. Subset Sum

Problem Statement

Determine whether a subset reaches target.

Example

[3,34,4,12,5,2],9 → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsets.
```

Java Optimized Solution

```
boolean[]d=new boolean[t+1];d[0]=true;for(int x:a)for(int s=t;s>=x;s--)d[s]|=d[s-x];return d[t];
```

53. Partition Equal Subset Sum

Problem Statement

Split array into equal-sum subsets.

Example

[1,5,11,5] → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsets.
```

Java Optimized Solution

```
int sum=0;for(int x:a)sum+=x;if((sum&1)==1)return false;return subset(a,sum/2);
```

54. Count Ways to Reach Target

Problem Statement

Count combinations that form target.

Example

coins [1,2,5],5 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive take/skip.
```

Java Optimized Solution

```
int[]d=new int[t+1];d[0]=1;for(int c:coins)for(int s=c;s<=t;s++)d[s]+=d[s-c];return d[t];
```

55. Coin Change

Problem Statement

Find minimum coins for an amount.

Example

[1,2,5],11 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive coin choices.
```

Java Optimized Solution

```
int[]d=new int[amt+1];Arrays.fill(d,amt+1);d[0]=0;for(int s=1;s<=amt;s++)for(int c:coins)if(c<=s)d[s]=Math.min(d[s],d[s-c]+1);return
```

56. Target Sum

Problem Statement

Assign +/− signs to reach target.

Example

[1,1,1,1,1],3 → 5

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate signs.
```

Java Optimized Solution

```
return targetDP(a,target);
```

57. Longest Common Subsequence

Problem Statement

Find LCS length of two strings.

Example

abcde,ace → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive take/skip.
```

Java Optimized Solution

```
int[][]d=new int[a.length()+1][b.length()+1];for(int i=1;i<=a.length();i++)for(int j=1;j<=b.length();j++)d[i][j]=a.charAt(i-1)==b.cha
```

58. Longest Common Substring

Problem Statement

Find longest common contiguous substring.

Example

ABABC,BABCA → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Compare every pair and extend.
```

Java Optimized Solution

```
int[][]d=new int[a.length()+1][b.length()+1];int best=0;for(int i=1;i<=a.length();i++)for(int j=1;j<=b.length();j++)if(a.charAt(i-1)=
```

59. Edit Distance

Problem Statement

Minimum insert/delete/replace edits.

Example

horse,ros → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive three operations.
```

Java Optimized Solution

```
int[][]d=new int[a.length()+1][b.length()+1];for(int i=0;i<=a.length();i++)d[i][0]=i;for(int j=0;j<=b.length();j++)d[0][j]=j;for(int
```

60. Minimum String Transformation Cost

Problem Statement

Transform one string into another with edit operations.

Example

cat,cut → 1

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive edits.
```

Java Optimized Solution

```
return editDistance(a,b);
```

61. Distinct Subsequences

Problem Statement

Count subsequences of s equal to t.

Example

rabbbit,rabbit → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate include/skip.
```

Java Optimized Solution

```
long[]d=new long[t.length()+1];d[0]=1;for(char c:s.toCharArray())for(int j=t.length();j>=1;j--)if(c==t.charAt(j-1))d[j]+=d[j-1];retu
```

62. LIS O(N²)

Problem Statement

Find longest increasing subsequence.

Example

[10,9,2,5,3,7,101,18] → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsequences.
```

Java Optimized Solution

```
int[]d=new int[a.length];Arrays.fill(d,1);int b=0;for(int i=0;i<a.length;i++){for(int j=0;j<i;j++)if(a[j]<a[i])d[i]=Math.max(d[i],d[
```

63. LIS O(N log N)

Problem Statement

Find LIS using tails and binary search.

Example

same → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Use O(N²) DP.
```

Java Optimized Solution

```
int[]t=new int[a.length];int n=0;for(int x:a){int l=0,r=n;while(l<r){int m=(l+r)/2;if(t[m]<x)l=m+1;else r=m;}t[l]=x;if(l==n)n++;}ret
```

64. Longest Divisible Subset

Problem Statement

Find largest pairwise-divisible subset length.

Example

[1,2,4,8] → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) DP after sorting.
```

Java Optimized Solution

```
Arrays.sort(a);int[]d=new int[a.length];Arrays.fill(d,1);int b=0;for(int i=0;i<a.length;i++){for(int j=0;j<i;j++)if(a[i]%a[j]==0)d[i
```

65. LIS with Difference Constraint

Problem Statement

Find LIS satisfying an extra difference bound.

Example

[1,3,5,7],D=2 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) DP.
```

Java Optimized Solution

```
return lisDiff(a,D);
```

66. Optimize O(N²) DP to O(N log N)

Problem Statement

Replace a quadratic transition with binary search/data structures.

Example

LIS → O(N log N)

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Use O(N²) transition.
```

Java Optimized Solution

```
return lisNlogN(a);
```

67. Space Optimization in DP

Problem Statement

Reduce DP memory while retaining the recurrence.

Example

LCS → O(min(n,m)) memory

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Use full table.
```

Java Optimized Solution

```
int[]p=new int[b.length()+1],c=new int[b.length()+1];for(int i=1;i<=a.length();i++){for(int j=1;j<=b.length();j++)c[j]=a.charAt(i-1)
```

68. Tabulation Optimization

Problem Statement

Convert slow recursion into iterative DP.

Example

Coin Change → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Use recursion.
```

Java Optimized Solution

```
return coinChange(coins,amt);
```

Trees, Graphs & Advanced Data Structures

69. Preorder Traversal

Problem Statement

Tree traversal Root-Left-Right.

Example

[1,2,3] → [1,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Stack/recursive traversal.
```

Java Optimized Solution

```
List<Integer>z=new ArrayList<>();Deque<TreeNode>s=new ArrayDeque<>();if(r!=null)s.push(r);while(!s.isEmpty()){TreeNode x=s.pop();z.a
```

70. Inorder Traversal

Problem Statement

Tree traversal Left-Root-Right.

Example

BST 2,1,3 → [1,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive traversal.
```

Java Optimized Solution

```
List<Integer>z=new ArrayList<>();Deque<TreeNode>s=new ArrayDeque<>();TreeNode c=r;while(c!=null||!s.isEmpty()){while(c!=null){s.push
```

71. Postorder Traversal

Problem Statement

Tree traversal Left-Right-Root.

Example

1 with children 2,3 → [2,3,1]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive traversal.
```

Java Optimized Solution

```
List<Integer>z=new ArrayList<>();post(r,z);return z;
```

72. Level Order Traversal

Problem Statement

Traverse a tree level by level.

Example

1/2,3 → [[1],[2,3]]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeated depth-first level scans.
```

Java Optimized Solution

```
List<List<Integer>>z=new ArrayList<>();if(r==null)return z;Queue<TreeNode>q=new ArrayDeque<>();q.add(r);while(!q.isEmpty()){int n=q.
```

73. Iterative DFS/BFS

Problem Statement

Traverse without recursive calls.

Example

Simple tree → traversal

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive DFS.
```

Java Optimized Solution

```
return preorder(r);
```

74. Tree Height

Problem Statement

Find maximum depth.

Example

[3,9,20,null,null,15,7] → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive height.
```

Java Optimized Solution

```
if(r==null)return 0;Queue<TreeNode>q=new ArrayDeque<>();q.add(r);int h=0;while(!q.isEmpty()){int n=q.size();h++;while(n-->0){TreeNod
```

75. Binary Tree Diameter

Problem Statement

Find longest path in edges.

Example

[1,2,3,4,5] → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Compute heights repeatedly.
```

Java Optimized Solution

```
int[]b={0};height(r,b);return b[0];
```

76. Weighted Tree Diameter

Problem Statement

Find maximum weighted tree path.

Example

0-1(3),1-2(4) → 7

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Run DFS from every node.
```

Java Optimized Solution

```
return weightedDiameter(g);
```

77. Maximum Path Sum

Problem Statement

Maximum sum path in a binary tree.

Example

[-10,9,20,null,null,15,7] → 42

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try paths through every node.
```

Java Optimized Solution

```
int[]b={Integer.MIN_VALUE};gain(r,b);return b[0];
```

78. LCA Binary Tree

Problem Statement

Lowest common ancestor of two nodes.

Example

LCA(5,1) → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Store root-to-node paths.
```

Java Optimized Solution

```
if(r==null||r==p||r==q)return r;TreeNode l=lca(r.left,p,q),x=lca(r.right,p,q);return l!=null&&x!=null?r:(l!=null?l:x);
```

79. LCA BST

Problem Statement

Use BST ordering for LCA.

Example

LCA(2,8) → 6

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
General tree LCA.
```

Java Optimized Solution

```
while(r!=null){if(p.val<r.val&&q.val<r.val)r=r.left;else if(p.val>r.val&&q.val>r.val)r=r.right;else return r;}return null;
```

80. LCA Binary Lifting

Problem Statement

Answer many LCA queries after preprocessing.

Example

Chain 0-1-2-3, LCA(3,1) → 1

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Walk parent chains.
```

Java Optimized Solution

```
// Precompute up[k][v]; lift the deeper node and then both nodes.
```

81. Subtree Sum

Problem Statement

Compute sum of every subtree.

Example

1/2,3 → root sum 6

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recompute subtree for each query.
```

Java Optimized Solution

```
return r==null?0:r.val+subtree(r.left)+subtree(r.right);
```

82. Subtree Queries

Problem Statement

Answer subtree sums quickly.

Example

leaf query → leaf value

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Traverse subtree per query.
```

Java Optimized Solution

```
// Euler tour + prefix/Fenwick interval query.
```

83. Maximum Independent Set on Tree

Problem Statement

Choose nodes with no parent-child pair.

Example

1 with 2,3 → 5

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate node subsets.
```

Java Optimized Solution

```
return Math.max(mis(r)[0],mis(r)[1]);
```

84. Minimum Operations on Tree

Problem Statement

Optimize node operations under a tree constraint.

Example

Exact constraint determines states.

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate operations.
```

Java Optimized Solution

```
// Tree DP with states for the local constraint.
```

85. Tree DP

Problem Statement

Compute optimal values from child states.

Example

House-robber tree → 5

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive enumeration.
```

Java Optimized Solution

```
int[]x=mis(r);return Math.max(x[0],x[1]);
```

86. Sum of Distances from Every Node

Problem Statement

Find distance sum for every root.

Example

0-1-2 → [3,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS from every node.
```

Java Optimized Solution

```
return allDistances(g);
```

87. DFS Rerooting

Problem Statement

Compute a value for every root.

Example

chain → [3,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Run DFS from every root.
```

Java Optimized Solution

```
return allDistances(g);
```

88. Graph Representation

Problem Statement

Build adjacency list.

Example

0-1,1-2 → adjacency list

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Adjacency matrix.
```

Java Optimized Solution

```
List<Integer>[]g=new ArrayList[n];for(int i=0;i<n;i++)g[i]=new ArrayList<>();for(int[]e:edges){g[e[0]].add(e[1]);g[e[1]].add(e[0]);}
```

89. Graph BFS

Problem Statement

Breadth-first traversal.

Example

0-1,0-2 → [0,1,2]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
DFS.
```

Java Optimized Solution

```
Queue<Integer>q=new ArrayDeque<>();List<Integer>z=new ArrayList<>();boolean[]v=new boolean[g.length];q.add(s);v[s]=true;while(!q.isEr
```

90. Graph DFS

Problem Statement

Depth-first traversal.

Example

0-1,0-2 → valid DFS

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeated search.
```

Java Optimized Solution

```
List<Integer>z=new ArrayList<>();Deque<Integer>s=new ArrayDeque<>();boolean[]v=new boolean[g.length];s.push(src);while(!s.isEmpty())
```

91. Connected Components

Problem Statement

Count connected components.

Example

5 nodes, edges 0-1,2-3 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeated scans.
```

Java Optimized Solution

```
int c=0;boolean[]v=new boolean[g.length];for(int i=0;i<g.length;i++)if(!v[i]){c++;bfs(g,i,v);}return c;
```

92. Cycle Detection Undirected BFS

Problem Statement

Detect an undirected cycle.

Example

triangle → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeated path search.
```

Java Optimized Solution

```
return cycleUndirectedBFS(g);
```

93. Cycle Detection Undirected DFS

Problem Statement

Detect cycle using parent tracking.

Example

triangle → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Path enumeration.
```

Java Optimized Solution

```
return cycleUndirectedDFS(g);
```

94. Cycle Detection Directed DFS

Problem Statement

Detect directed cycle using recursion states.

Example

0→1→2→0 → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate paths.
```

Java Optimized Solution

```
return directedCycle(g);
```

95. Cycle Detection Kahn

Problem Statement

A directed graph has a cycle if topo count < n.

Example

cycle → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeated indegree scans.
```

Java Optimized Solution

```
return cycleKahn(g);
```

96. Bipartite BFS

Problem Statement

2-color graph with BFS.

Example

square → true; triangle → false

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try color assignments.
```

Java Optimized Solution

```
return bipartiteBFS(g);
```

97. Bipartite DFS

Problem Statement

2-color graph recursively.

Example

triangle → false

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try all assignments.
```

Java Optimized Solution

```
return bipartiteDFS(g);
```

98. Topological Sort DFS

Problem Statement

Order DAG vertices by DFS finish time.

Example

0→1→2 → [0,1,2]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try permutations.
```

Java Optimized Solution

```
return topoDFS(g);
```

99. Topological Sort Kahn

Problem Statement

Order DAG using indegrees.

Example

0→1→2 → [0,1,2]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeatedly find zero indegree.
```

Java Optimized Solution

```
return topoKahn(g);
```

100. Course Schedule

Problem Statement

Determine if dependency graph is acyclic.

Example

cycle → false

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try course orders.
```

Java Optimized Solution

```
return topoKahn(g).size()==g.length;
```

101. Shortest Path Unweighted

Problem Statement

Shortest edge distance from source.

Example

0-1-2 → [0,1,2]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS per destination.
```

Java Optimized Solution

```
return shortestUnweighted(g,s);
```

102. Multi-source BFS

Problem Statement

Distance from nearest source.

Example

sources 0,3 on path → [0,1,1,0]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS from every source separately.
```

Java Optimized Solution

```
return multiBFS(g,sources);
```

103. Dijkstra

Problem Statement

Shortest paths with nonnegative weights.

Example

0-2(1),2-1(2) → d1=3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeatedly select minimum unvisited node.
```

Java Optimized Solution

```
return dijkstra(g,s);
```

104. Weighted Shortest Path

Problem Statement

Find minimum cost path in a nonnegative graph.

Example

weighted graph → minimum distance

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate paths.
```

Java Optimized Solution

```
return dijkstra(g,s);
```

105. Network Delay

Problem Statement

Maximum shortest distance from a source.

Example

2→1(1),2→3(1),3→4(1) → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Run shortest path to every node.
```

Java Optimized Solution

```
long[]d=dijkstra(g,s);long b=0;for(long x:d){if(x==Long.MAX_VALUE)return -1;b=Math.max(b,x);}return b;
```

106. Bellman-Ford

Problem Statement

Shortest paths with negative edges.

Example

0→1(4),1→2(-2) → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Relax all edges repeatedly.
```

Java Optimized Solution

```
return bellmanFord(n,e,s);
```

107. Floyd-Warshall

Problem Statement

All-pairs shortest paths.

Example

3-node weighted graph → distance matrix

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate intermediate paths.
```

Java Optimized Solution

```
for(int k=0;k<d.length;k++)for(int i=0;i<d.length;i++)for(int j=0;j<d.length;j++)if(d[i][k]!=Long.MAX_VALUE&&d[k][j]!=Long.MAX_VALUE
```

108. Shortest Path in DAG

Problem Statement

Use topological order for DAG shortest paths.

Example

0→1(2),1→2(1) → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate paths.
```

Java Optimized Solution

```
return dagShortest(g,s);
```

109. MST Kruskal

Problem Statement

Minimum spanning tree using sorting + DSU.

Example

triangle weights 1,2,3 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate edge subsets.
```

Java Optimized Solution

```
return mstKruskal(n,e);
```

110. MST Prim

Problem Statement

Minimum spanning tree using a priority queue.

Example

triangle → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate spanning trees.
```

Java Optimized Solution

```
return mstPrim(n,e);
```

111. Connect All Cities

Problem Statement

Minimum cost to connect every city.

Example

weighted city graph → MST cost

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate spanning trees.
```

Java Optimized Solution

```
return mstKruskal(n,e);
```

112. DSU Basics

Problem Statement

Support union and find efficiently.

Example

union 0-1,1-2 → same component

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Naive parent chains.
```

Java Optimized Solution

```
DSU d=new DSU(n);d.union(a,b);return d.find(a);
```

113. Connected Components DSU

Problem Statement

Count components after edge additions.

Example

5 nodes, 2 edges → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS/DFS components.
```

Java Optimized Solution

```
DSU d=new DSU(n);int c=n;for(int[]e:edges)if(d.union(e[0],e[1]))c--;return c;
```

114. Cycle Detection DSU

Problem Statement

Detect cycle while adding undirected edges.

Example

triangle → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
DFS cycle detection.
```

Java Optimized Solution

```
DSU d=new DSU(n);for(int[]e:edges)if(!d.union(e[0],e[1]))return true;return false;
```

115. Connectivity Queries

Problem Statement

Answer whether two nodes are connected.

Example

union 0-1,1-2; query 0,2 → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS per query.
```

Java Optimized Solution

```
return d.find(a)==d.find(b);
```

116. Dynamic Connectivity

Problem Statement

Process edge additions and connectivity queries.

Example

add 0-1, query 0-1 → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Rebuild components.
```

Java Optimized Solution

```
return d.find(a)==d.find(b);
```

117. Fenwick Point Update Prefix Query

Problem Statement

Point updates and prefix sums in O(logN).

Example

[1,2,3], add 5 at 1 → prefix1=8

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recompute prefix sums.
```

Java Optimized Solution

```
for(i++;i<=n;i+=i&-i)bit[i]+=v;
```

118. Fenwick Range Sum

Problem Statement

Dynamic range sums with point updates.

Example

[1,2,3,4], [1,3] → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Sum every range.
```

Java Optimized Solution

```
return bit.sum(r)-bit.sum(l-1);
```

119. Fenwick Frequency Queries

Problem Statement

Maintain frequencies and prefix counts.

Example

[1,2,2,4], <=2 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Count values directly.
```

Java Optimized Solution

```
return bit.sum(x);
```

120. Segment Tree Range Sum

Problem Statement

Dynamic range sum with point update.

Example

[1,3,5], [0,2] → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan range.
```

Java Optimized Solution

```
return seg.query(l,r);
```

121. Segment Tree Range Minimum

Problem Statement

Dynamic range minimum.

Example

[5,2,7,1], [1,3] → 1

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan range.
```

Java Optimized Solution

```
return minSeg.query(l,r);
```

122. Segment Tree Range Maximum

Problem Statement

Dynamic range maximum.

Example

[5,2,7,1], [1,3] → 7

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan range.
```

Java Optimized Solution

```
return maxSeg.query(l,r);
```

123. Segment Tree Point Update

Problem Statement

Point update plus range query.

Example

update index 1 to 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Modify then scan.
```

Java Optimized Solution

```
seg.update(i,v);return seg.query(l,r);
```

124. Segment Tree Lazy Range Update

Problem Statement

Range update and range sum.

Example

[1,2,3,4], add5 [1,2] → 20

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Update every element.
```

Java Optimized Solution

```
lazy.add(l,r,v);return lazy.query(0,n-1);
```

125. Dynamic Range Min/Max

Problem Statement

Dynamic range extrema.

Example

query after update → current extrema

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan every query.
```

Java Optimized Solution

```
return seg.query(l,r);
```

126. Lazy Propagation

Problem Statement

Push deferred range updates only when needed.

Example

range add + query → correct sum

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Apply update to every element.
```

Java Optimized Solution

```
lazy.add(l,r,v);
```

127. Multiple Dynamic Range Queries

Problem Statement

Choose BIT or segment tree based on operation type.

Example

mixed updates/queries → correct results

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Perform each operation directly.
```

Java Optimized Solution

```
// BIT for point updates; lazy segment tree for range updates.
```

128. Frequency Query Using Fenwick

Problem Statement

Count values ≤ x dynamically.

Example

[1,2,2,4], ≤2 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan all values.
```

Java Optimized Solution

```
return bit.sum(x);
```

129. Order Statistic Query

Problem Statement

Find k-th smallest from frequencies.

Example

[1,2,2,5], k=3 → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Sort values.
```

Java Optimized Solution

```
// Fenwick binary lifting returns the k-th rank.
```

130. Coordinate Compression + BIT

Problem Statement

Map large values to compact ranks.

Example

[10,30,20] → [1,3,2]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Compare every pair.
```

Java Optimized Solution

```
// Sort unique values, map each value to its rank.
```

131. Hashing + Sliding Window

Problem Statement

Use frequency map in a constrained window.

Example

[1,2,1,3], k=2 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate windows.
```

Java Optimized Solution

```
return atMostK(a,k);
```

132. Longest Valid Subarray

Problem Statement

Find longest window satisfying a monotonic condition.

Example

[1,2,1,2,3],k=2 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate all windows.
```

Java Optimized Solution

```
return atMostK(a,k);
```

133. Frequency-Constrained Window

Problem Statement

Find a window satisfying frequency constraints.

Example

unique substring → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate substrings.
```

Java Optimized Solution

```
return longestUnique(s);
```

134. Minimum Feasible Value

Problem Statement

Find smallest feasible answer with monotonic check.

Example

ship capacity → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try every answer.
```

Java Optimized Solution

```
return binaryMin(lo,hi,ok);
```

135. Maximum Achievable Value

Problem Statement

Find largest feasible answer.

Example

aggressive cows → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try every answer.
```

Java Optimized Solution

```
return binaryMax(lo,hi,ok);
```

136. Binary Search on Answer + Greedy

Problem Statement

Binary search answer and greedily validate.

Example

aggressive cows → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try every answer.
```

Java Optimized Solution

```
return aggressive(a,k);
```

137. LIS + Binary Search

Problem Statement

Optimize LIS to O(NlogN).

Example

standard LIS → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) LIS.
```

Java Optimized Solution

```
return lisNlogN(a);
```

138. Optimized DP + Binary Search

Problem Statement

Use binary search to reduce a DP transition.

Example

LIS → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) DP.
```

Java Optimized Solution

```
return lisNlogN(a);
```

139. DP State Optimization

Problem Statement

Keep only necessary DP states.

Example

House Robber → 12

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Full DP array.
```

Java Optimized Solution

```
int p2=0,p1=0;for(int x:a){int c=Math.max(p1,p2+x);p2=p1;p1=c;}return p1;
```

140. Tree as Undirected Graph

Problem Statement

Represent a tree as an adjacency list.

Example

0-1,1-2 → graph

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Matrix representation.
```

Java Optimized Solution

```
return adjacencyList(n,e);
```

141. Tree Diameter Two BFS/DFS

Problem Statement

Find tree diameter using two traversals.

Example

0-1-2-3 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS from every node.
```

Java Optimized Solution

```
int a=far(g,0).node;return far(g,a).dist;
```

142. Tree Queries with DFS Precomputation

Problem Statement

Flatten subtree intervals for fast queries.

Example

subtree query → interval sum

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Traverse subtree each time.
```

Java Optimized Solution

```
// Euler tour + prefix/Fenwick.
```

143. LCA + Binary Lifting

Problem Statement

Answer many LCA queries in O(logN).

Example

chain LCA(3,1) → 1

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Parent path search.
```

Java Optimized Solution

```
// up[k][v] ancestor table + binary lifting.
```

144. Rerooting DP

Problem Statement

Calculate a tree value for every root.

Example

0-1-2 → [3,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS from every root.
```

Java Optimized Solution

```
return allDistances(g);
```

Infosys SP Mixed & Final Must-Master

145. Greedy + Arrays Selection

Problem Statement

Select/rearrange elements under a greedy condition.

Example

array selection → optimum

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate selections.
```

Java Optimized Solution

```
// Sort + greedy invariant.
```

146. Prefix Sum / Pre-computation

Problem Statement

Precompute values for repeated queries.

Example

range sum → O(1) query

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan every query.
```

Java Optimized Solution

```
// Prefix array.
```

147. HashMap / Frequency Query

Problem Statement

Answer repeated frequency queries.

Example

freq(1) in [1,2,1] → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan array per query.
```

Java Optimized Solution

```
// HashMap frequencies.
```

148. Sliding Window

Problem Statement

Find longest/shortest valid contiguous window.

Example

abcabcbb → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate substrings.
```

Java Optimized Solution

```
return longestUnique(s);
```

149. Advanced Array Manipulation

Problem Statement

Optimize a non-trivial array transformation.

Example

constraint-specific

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate all arrangements.
```

Java Optimized Solution

```
// Identify sorting/two-pointer/greedy invariant.
```

150. DP Optimized Tabulation

Problem Statement

Convert exponential recursion into iterative DP.

Example

LCS → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursion.
```

Java Optimized Solution

```
return lcs(a,b);
```

151. DP O(NlogN) Optimization

Problem Statement

Use binary search/data structures for DP.

Example

LIS → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) DP.
```

Java Optimized Solution

```
return lisNlogN(a);
```

152. Binary Search on Answer

Problem Statement

Search monotonic answer space.

Example

shipping → minimum capacity

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Linear answer scan.
```

Java Optimized Solution

```
return ship(w,d);
```

153. Segment Tree / Fenwick Problem

Problem Statement

Choose data structure for dynamic queries.

Example

point update + range sum

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Direct range scan.
```

Java Optimized Solution

```
// Fenwick or segment tree.
```

154. Hard Tree Algorithm

Problem Statement

Solve a hard tree optimization/query problem.

Example

tree-specific

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate possibilities.
```

Java Optimized Solution

```
// DFS/tree DP/LCA/rerooting.
```

155. Tree DP / Rerooting

Problem Statement

Compute optimized value across all roots.

Example

distance sums → [3,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS from every root.
```

Java Optimized Solution

```
return allDistances(g);
```

156. Hard Graph Optimization

Problem Statement

Optimize a weighted graph problem.

Example

MST/shortest path

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate paths/trees.
```

Java Optimized Solution

```
// Dijkstra/MST/DSU depending on statement.
```

157. Advanced Data Structure

Problem Statement

Process dynamic range/frequency operations.

Example

range query/update

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Direct updates.
```

Java Optimized Solution

```
// Fenwick/segment tree.
```

158. Segment Tree Lazy Propagation

Problem Statement

Range update + range query.

Example

add5 to [1,2]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Update each element.
```

Java Optimized Solution

```
// LazySegTree.
```

159. DSU Connectivity

Problem Statement

Online connectivity under edge additions.

Example

add 0-1, query → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS after each update.
```

Java Optimized Solution

```
// DSU with path compression.
```

160. Maximum Subarray Sum — Kadane

Problem Statement

Find maximum contiguous sum.

Example

[-2,1,-3,4,-1,2,1] → 6

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) subarrays.
```

Java Optimized Solution

```
int c=a[0],b=a[0];for(int i=1;i<a.length;i++){c=Math.max(a[i],c+a[i]);b=Math.max(b,c);}return b;
```

161. LCS

Problem Statement

Find longest common subsequence length.

Example

abcde,ace → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive subsequences.
```

Java Optimized Solution

```
return lcs(a,b);
```

162. LIS

Problem Statement

Find longest increasing subsequence.

Example

10,9,2,5,3,7 → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) DP.
```

Java Optimized Solution

```
return lisNlogN(a);
```

163. Edit Distance

Problem Statement

Minimum string transformation edits.

Example

horse,ros → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive edits.
```

Java Optimized Solution

```
return editDistance(a,b);
```

164. 0/1 Knapsack

Problem Statement

Maximum value under capacity.

Example

C=7 → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate subsets.
```

Java Optimized Solution

```
return knapsack(w,v,C);
```

165. Coin Change

Problem Statement

Minimum coins for amount.

Example

[1,2,5],11 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recursive choices.
```

Java Optimized Solution

```
return coinChange(c,amount);
```

166. Sliding Window + HashMap

Problem Statement

Longest valid window.

Example

abcabcbb → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate windows.
```

Java Optimized Solution

```
return longestUnique(s);
```

167. Prefix Sum + Range Queries

Problem Statement

Answer static range sums.

Example

[1,2,3,4],[1,3] → 9

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan ranges.
```

Java Optimized Solution

```
// prefix sum.
```

168. Greedy Interval Scheduling

Problem Statement

Maximum non-overlapping intervals.

Example

(1,2),(2,3) → 2

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Subset enumeration.
```

Java Optimized Solution

```
return activitySelection(a);
```

169. Binary Search on Answer

Problem Statement

Find optimal feasible value.

Example

aggressive cows → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Linear answer search.
```

Java Optimized Solution

```
return aggressive(a,k);
```

170. Tree Diameter

Problem Statement

Longest tree path.

Example

chain 4 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS/DFS from every node.
```

Java Optimized Solution

```
return treeDiameter(g);
```

171. Lowest Common Ancestor

Problem Statement

Lowest common ancestor.

Example

LCA(5,1) → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Store paths.
```

Java Optimized Solution

```
return lca(r,p,q);
```

172. Subtree Sum / Tree DP

Problem Statement

Compute subtree aggregate.

Example

1/2,3 → 6

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Recompute subtree.
```

Java Optimized Solution

```
return subtree(r);
```

173. Tree Rerooting

Problem Statement

Compute root-dependent value for every node.

Example

0-1-2 → [3,2,3]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS from every root.
```

Java Optimized Solution

```
return allDistances(g);
```

174. Cycle Detection

Problem Statement

Detect graph cycle.

Example

triangle → true

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Search paths.
```

Java Optimized Solution

```
return cycleUndirectedDFS(g);
```

175. Bipartite Graph

Problem Statement

Determine 2-colorability.

Example

triangle → false

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try colorings.
```

Java Optimized Solution

```
return bipartiteBFS(g);
```

176. Topological Sort

Problem Statement

Order a DAG.

Example

0→1→2 → [0,1,2]

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Try permutations.
```

Java Optimized Solution

```
return topoKahn(g);
```

177. Dijkstra

Problem Statement

Shortest paths with nonnegative weights.

Example

weighted graph → distances

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Repeated minimum scan.
```

Java Optimized Solution

```
return dijkstra(g,s);
```

178. Kruskal + DSU

Problem Statement

Minimum spanning tree.

Example

weights 1,2,3 → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate spanning trees.
```

Java Optimized Solution

```
return mstKruskal(n,e);
```

179. Prim MST

Problem Statement

Minimum spanning tree.

Example

triangle → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Enumerate spanning trees.
```

Java Optimized Solution

```
return mstPrim(n,e);
```

180. DSU Connectivity

Problem Statement

Maintain connected components.

Example

union 0-1,1-2 → connected

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
BFS per query.
```

Java Optimized Solution

```
return d.find(a)==d.find(b);
```

181. Fenwick Tree

Problem Statement

Dynamic point update + prefix/range sum.

Example

add5 then query → updated sum

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan each query.
```

Java Optimized Solution

```
// Fenwick.
```

182. Segment Tree

Problem Statement

Dynamic range query/update.

Example

range sum/min/max

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Scan each query.
```

Java Optimized Solution

```
// Segment tree.
```

183. Lazy Segment Tree

Problem Statement

Range updates + range queries.

Example

range add → updated sum

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Update every element.
```

Java Optimized Solution

```
// Lazy propagation.
```

184. Hashing / Frequency Maps

Problem Statement

Count and locate values quickly.

Example

[1,2,1] → freq map

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Nested counting.
```

Java Optimized Solution

```
// HashMap.
```

185. Advanced Array Optimization

Problem Statement

Choose optimal array pattern.

Example

Two Sum → linear

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Nested pairs.
```

Java Optimized Solution

```
// Hashing/two pointers/prefix.
```

186. DP Tabulation Optimization

Problem Statement

Optimize recursion into iterative DP.

Example

LCS → 3

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
Exponential recursion.
```

Java Optimized Solution

```
return lcs(a,b);
```

187. O(N²) → O(NlogN) Optimization

Problem Statement

Recognize binary-search/data-structure optimization.

Example

LIS → 4

Sample Test Cases

- Example input → expected output based on the example.
- Edge case → verify empty/single-element/minimum-size input where applicable.

Java Brute Force Code

```
O(N²) LIS.
```

Java Optimized Solution

```
return lisNlogN(a);
```

Canonical Advanced Implementations

Fenwick Tree

```
static class Fenwick{int n;long[] bit;Fenwick(int n){this.n=n;bit=new long[n+1];}void add(int i,long v){for(i++;i<=n;i+=i&-i)bit[i]+
```

DSU / Union-Find

```
static class DSU{int[] p,sz;DSU(int n){p=new int[n];sz=new int[n];for(int i=0;i<n;i++){p[i]=i;sz[i]=1;}}int find(int x){return p[x]=
```

Placement Revision Rule

Study the problems in numerical order. First understand the brute-force idea, then identify the bottleneck, then memorize the optimized pattern and finally code it from scratch without looking.